

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0508

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Code Challenge (High)

Assessment Tool Weightage: 35%

Faculty Name: Dr . G . Ayyappa n

Student Reg. No: 192524326

Name: Pooja.S

QUESTIONS				Total Marks: 50
Q.No	Question	Bloom's Level	Marks	
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5	
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5	
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5	
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5	
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5	
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5	
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5	
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5	

9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I Pooja(192524326) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Heap File Organization — Insert, Delete and Sequential Scan

```
records = []
def insert(record):
    records.append(record)
    print("Record inserted:", record)
def delete(record):
    if record in records:
        records.remove(record)
        print("Record deleted:", record)
    else:
        print("Record not found")
def display():
    print("All Records:")
    for r in records:
        print(r)
insert(10)
insert(20)
insert(30)
insert(40)
display()
delete(20)
display()
```

Output:

```
Record inserted: 10
Record inserted: 20
Record inserted: 30
Record inserted: 40
All Records:
10
20
30
40
Record deleted: 20
All Records:
10
30
40
```

2. Static Hashing with Chaining — 500 Records

```
SIZE = 10
hash_table = [[] for i in range(SIZE)]
def insert(key):
    index = key % SIZE
    hash_table[index].append(key)
for key in [101, 111, 121, 102, 112, 122, 103]:
    insert(key)
print("Hash Table:")
for i in range(SIZE):
    print(i, ":", hash_table[i])
```

Output:

```
Hash Table:
0 : []
1 : [101, 111, 121]
2 : [102, 112, 122]
3 : [103]
4 : []
5 : []
6 : []
7 : []
8 : []
9 : []
```

3. Sorted File Organization + Binary Search

```
records = [10, 20, 30, 40, 50, 60, 70]
def binary_search(key):
    low = 0
    high = len(records) - 1
    while low <= high:
        mid = (low + high) // 2
        if records[mid] == key:
            return mid
        elif records[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
key = 40
result = binary_search(key)
if result != -1:
    print("Record found at position:", result)
else:
    print("Record not found")
```

Output:

Record found at position: 3

4. B-Tree of Order 4 — Insert, Search and Display

```
class BTree:
    def __init__(self):
        self.keys = []
    def insert(self, key):
        self.keys.append(key)
        self.keys.sort()
    def search(self, key):
        if key in self.keys:
            print(key, "found")
        else:
            print(key, "not found")
    def display(self):
        print("B-Tree keys:", self.keys)
tree = BTree()
keys = [10, 5, 6, 12, 30, 7, 17]
for key in keys:
    tree.insert(key)
tree.display()
tree.search(12)
tree.search(50)|
```

Output:

B-Tree keys: [5, 6, 7, 10, 12, 17, 30]

12 found

50 not found

5. B+ Tree with Leaf-Level Linking

```
class Leaf:
    def __init__(self, keys):
        self.keys = keys
        self.next = None
leaf1 = Leaf([10, 20, 30])
leaf2 = Leaf([40, 50, 60])
leaf3 = Leaf([70, 80, 90])
leaf1.next = leaf2
leaf2.next = leaf3
current = leaf1
print("B+ Tree Leaf Nodes:")
while current:
    print(current.keys)
    current = current.next
```

Output:

```
B+ Tree Leaf Nodes:
[10, 20, 30]
[40, 50, 60]
[70, 80, 90]
```

6. Dense Primary Index on a Sorted File

```
records = [  
    (10, "A"),  
    (20, "B"),  
    (30, "C"),  
    (40, "D"),  
    (50, "E")  
]  
index = {}  
for position, record in enumerate(records):  
    key = record[0]  
    index[key] = position  
print("Primary Index:")  
for key, position in index.items():  
    print(key, "->", position)  
key = 30  
if key in index:  
    position = index[key]  
    print("Record found:", records[position])  
else:  
    print("Record not found")
```

Output:

```
Primary Index:  
10 -> 0  
20 -> 1  
30 -> 2  
40 -> 3  
50 -> 4  
  
Record found: (30, 'C')
```


7. Extendible Hashing — Directory Doubling

```
directory = {
    0: [],
    1: []
}
bucket_size = 2
records = [1, 3, 5, 7, 2, 4, 6, 8]
for key in records:
    index = key % len(directory)
    if len(directory[index]) < bucket_size:
        directory[index].append(key)
    else:
        old = directory
        directory = {0: [], 1: [], 2: [], 3: []}
        for values in old.values():
            for value in values:
                new_index = value % 4
                directory[new_index].append(value)
        new_index = key % 4
        directory[new_index].append(key)
print("Final Directory:")
for index in directory:
    print(index, ":", directory[index])
```

Output:

```
Final Directory:
0 : [4, 8]
1 : [1, 5]
2 : [2, 6]
3 : [3, 7]
```

8. Secondary Storage Block Access — Sequential vs Indexed Retrieval

```
records = 100
records_per_block = 10
blocks = records // records_per_block
sequential_io = blocks
index_io = 1          # access index
data_io = 1           # access data block
indexed_io = index_io + data_io
print("Total Records:", records)
print("Records per Block:", records_per_block)
print("Total Blocks:", blocks)
print("Sequential Retrieval I/O:", sequential_io)
print("Indexed Retrieval I/O:", indexed_io)
```

Output:

```
Total Records: 100
Records per Block: 10
Total Blocks: 10
Sequential Retrieval I/O: 10
Indexed Retrieval I/O: 2
```

9. Sparse Index vs Dense Index with Timing Analysis

```
import time
records = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
dense_index = {}
for i in range(len(records)):
    dense_index[records[i]] = i
sparse_index = {}
for i in range(0, len(records), 2):
    sparse_index[records[i]] = i
key = 70
start = time.time()
if key in dense_index:
    position = dense_index[key]
dense_time = time.time() - start
start = time.time()
position = -1
for k in sparse_index:
    if k <= key:
        position = sparse_index[k]
for i in range(position, len(records)):
    if records[i] == key:
        position = i
        break
sparse_time = time.time() - start
print("Records:", records)
print("\nDense Index:")
print(dense_index)
print("\nSparse Index:")
print(sparse_index)
print("\nSearch Key:", key)
print("Dense Index Time:", dense_time)
print("Sparse Index Time:", sparse_time)
```

Output:

Records: [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

Dense Index:

{10: 0, 20: 1, 30: 2, 40: 3, 50: 4, 60: 5, 70: 6, 80: 7, 90: 8, 100: 9}

Sparse Index:

{10: 0, 30: 2, 50: 4, 70: 6, 90: 8}

Search Key: 70

Dense Index Time: 0.000001

Sparse Index Time: 0.000002

10. B+ Tree — Range Query 15 to 45

```
class Leaf:
    def __init__(self, keys):
        self.keys = keys
        self.next = None
leaf1 = Leaf([5, 10, 15, 20])
leaf2 = Leaf([25, 30, 35, 40])
leaf3 = Leaf([45, 50, 55, 60])
leaf1.next = leaf2
leaf2.next = leaf3
print("B+ Tree:")
current = leaf1
while current:
    print(current.keys)
    current = current.next
low = 15
high = 45
print("\nRecords between 15 and 45:")
current = leaf1
while current:
    for key in current.keys:
        if low <= key <= high:
            print(key, end=" ")
    current = current.next
```

Output:

B+ Tree:

[5, 10, 15, 20]

[25, 30, 35, 40]

[45, 50, 55, 60]

Records between 15 and 45:

15 20 25 30 35 40 45